

<https://www.geeksforgeeks.org/operating-systems/process-creation-and-deletions-in-operating-systems/>

<https://bhanuprakashhegala.hashnode.dev/process-creation-in-linux?>

<https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa?>

[Book: xv6: a simple, Unix-like teaching operating system.](#)

Process Creation in Linux and Windows 10/11

In operating systems, process creation is a fundamental concept that allows for multitasking and efficient resource utilization. While the underlying principles remain consistent, the mechanisms differ between operating systems.

Both Linux and Windows provide mechanisms for process creation and program execution, their main difference is in implementation. Linux's `fork()` and `exec()` offer flexibility allowing manipulation of the new process before executing a new program. Whereas, Windows uses one process with `CreateProcess()`, handling both creation and execution in a single step.

Linux Process Creation: `fork()` and `exec()`

In Linux, the creation of a new process typically involves two system calls: `fork()` and `exec()`.

fork(): This system call creates a new process by duplicating the calling process. The new process, known as the child process, is an exact copy of the parent process, including its memory, file descriptors, and signal handlers. The only difference is the return value of `fork()`, which is 0 in the child process and the child's PID in the parent process. After a `fork()`, the child process may replace its memory space with a new program using the `exec()` family of functions. This overlays the current process image with a new one, effectively running a different program within the same process ID.

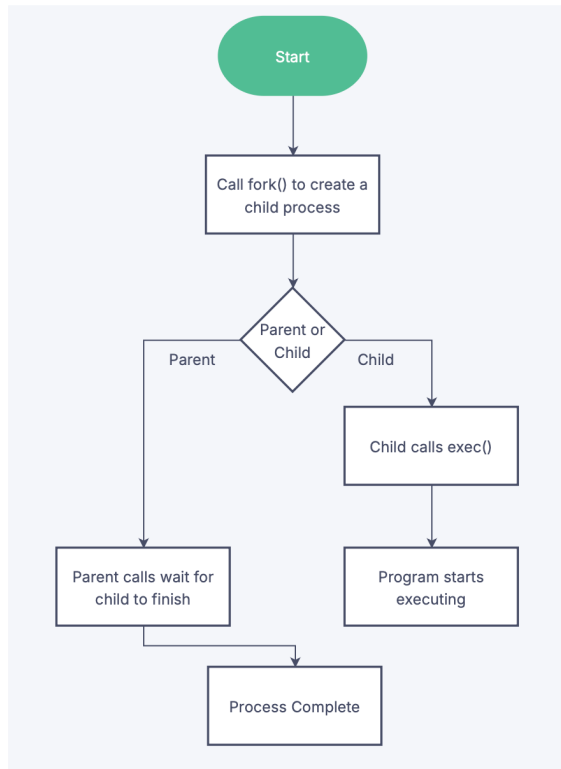
Windows 10/11 Process Creation:

Windows uses a different approach to do process creation through the `CreateProcess()` function.

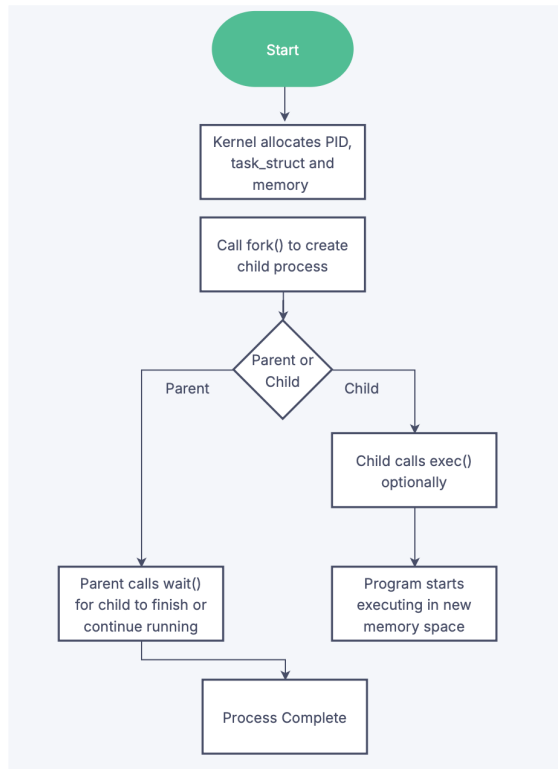
This single function creates a new process and primary thread. It loads the executable into the new process's memory space and begins execution. The function requires a extensive number of parameters, including executable name, command line arguments, security features, environment variables, and i/o directions.

Unlike the Linux model, Windows does not have a direct equivalent to `fork()`. Instead, `CreateProcess()` handles both process creation and program execution in one step.

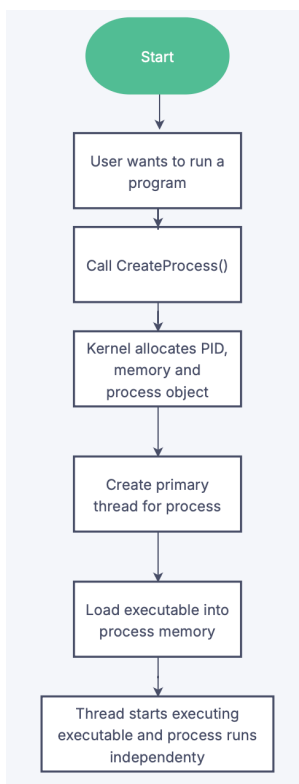
1. xv6 Process Creation



2. Linux Process Creation



3. Windows 10/11 Process Creation



Feature	xv6 (fork + exec)	Linux 6.0+ (fork/clone + execve)	Windows 10/11 (CreateProcess)
System Call(s)	fork(), exec()	fork()/clone(), execve()	CreateProcess()
Process Creation	Copy parent, then exec	Copy-on-write, then execve	One-step creation + execution
Threads	N/A (single-threaded)	clone() supports threads	Primary thread created by default
Parent-Child Relation	Yes	Yes	No direct relation
Resource Handling	Inherits descriptors	Inherits descriptors	Uses handles/objects

^Comparison Table